GitHub Copilot Power User & Erweiterungen

Harald Binkle
FullStacker & Trainer

Nico Orschel
DevOps Whisperer

Agenda

0. Introduction and Overview

1. Power user functions and effective prompt commands

2. Optimization of GitHub Copilot through extensions

3. Extension via MCP (client / server)

4. Q&A and discussion

Über uns

Harald Binkle

Full Stack | DevOps | Consultant

Harald.Binkle@xebia.com

Nico Orschel

DevOps | Consultant

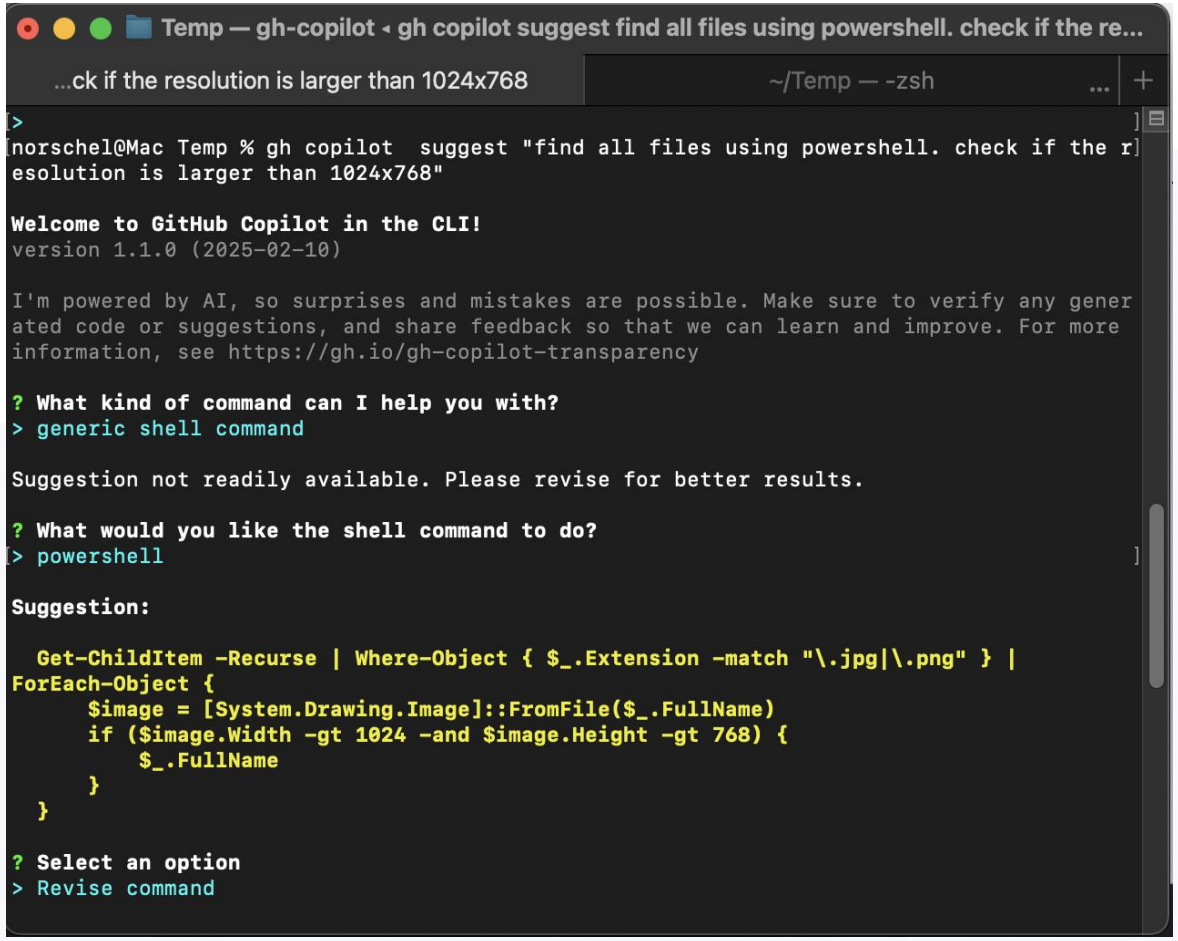
Nico.Orschel@xebia.com

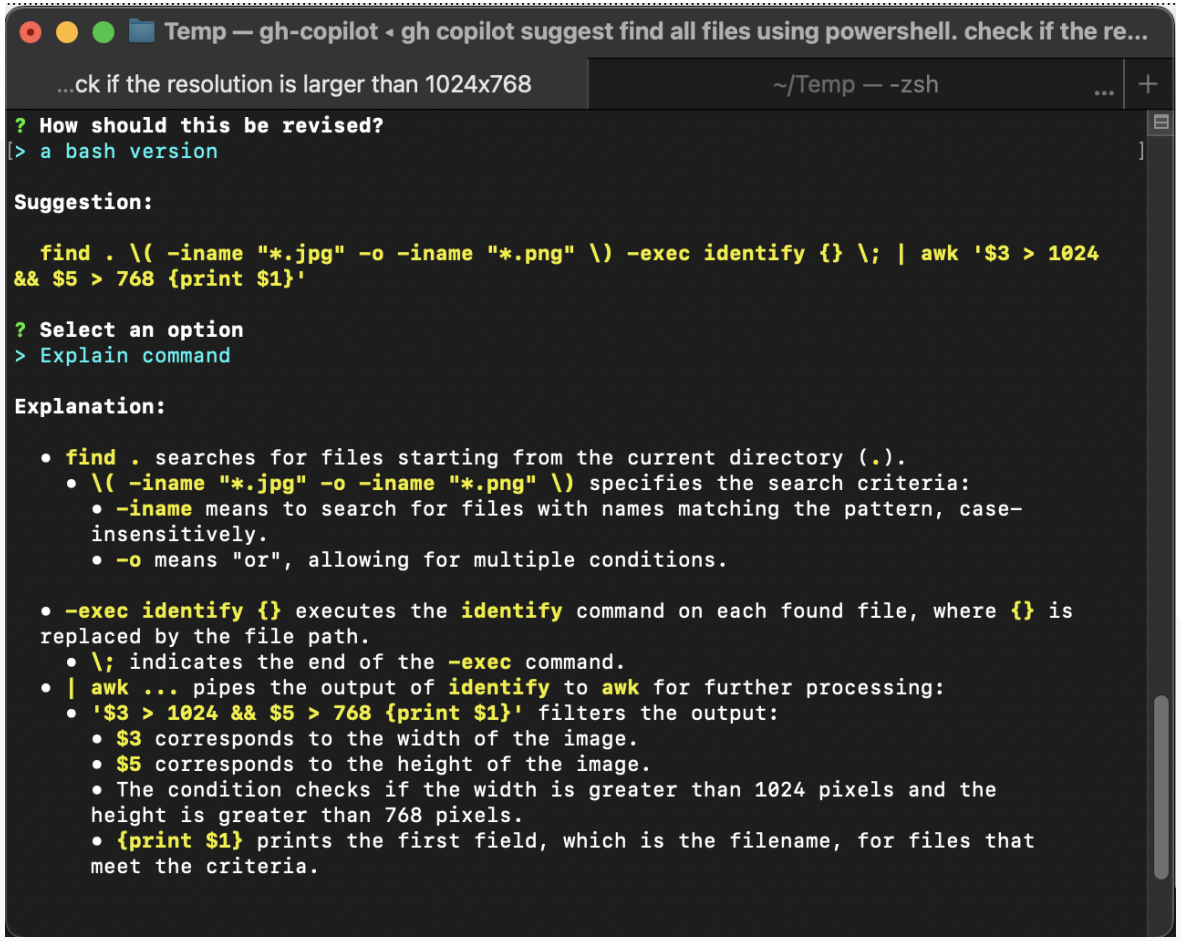
1. Power user functions and effective prompt commands

Using the gh copilot commands on the CLI

- Overview:
  - The **gh copilot** command is part of the GitHub CLI, enabling seamless integration with GitHub Copilot directly from your terminal.
  - It allows developers to generate code suggestions, explanations, and even entire files without leaving the command line.
- Key Features:
  - Code Generation: Instantly generate code snippets or functions based on natural language prompts.
  - Code Explanation: Get explanations for code blocks to understand logic and intent.
  - Seamless Workflow: Integrates with your existing CLI workflow, boosting productivity.

Demo


Temp — gh-copilot ◂ gh copilot suggest find all files using powershell. check if the re...
...ck if the resolution is larger than 1024x768
~/Temp — -zsh
norschel@Mac Temp % gh copilot suggest "find all files using powershell. check if the resolution is larger than 1024x768"
Welcome to GitHub Copilot in the CLI!
version 1.1.0 (2025-02-10)
I'm powered by AI, so surprises and mistakes are possible. Make sure to verify any generated code or suggestions, and share feedback so that we can learn and improve. For more information, see https://gh.io/gh-copilot-transparency
? What kind of command can I help you with?
> generic shell command
Suggestion not readily available. Please revise for better results.
? What would you like the shell command to do?
> powershell
Suggestion:
Get-ChildItem -Recurse | Where-Object { $_.Extension -match "\.jpg|\.png" } | ForEach-Object {
$image = [System.Drawing.Image]::FromFile($_.FullName)
if ($image.Width -gt 1024 -and $image.Height -gt 768) {
$_.FullName
}
}
? Select an option
> Revise command


Temp — gh-copilot ◂ gh copilot suggest find all files using powershell. check if the re...
...ck if the resolution is larger than 1024x768
~/Temp — -zsh
? How should this be revised?
> a bash version
Suggestion:
find . \(-iname "*.jpg" -o -iname "*.png" \) -exec identify {} \; | awk '$3 > 1024 && $5 > 768 {print $1}'
? Select an option
> Explain command
Explanation:
• find . searches for files starting from the current directory (.).
• \(-iname "*.jpg" -o -iname "*.png" \) specifies the search criteria:
• -iname means to search for files with names matching the pattern, case-insensitively.
• -o means "or", allowing for multiple conditions.
• -exec identify {} executes the identify command on each found file, where {} is replaced by the file path.
• \; indicates the end of the -exec command.
• | awk ... pipes the output of identify to awk for further processing:
• '$3 > 1024 && $5 > 768 {print $1}' filters the output:
• $3 corresponds to the width of the image.
• $5 corresponds to the height of the image.
• The condition checks if the width is greater than 1024 pixels and the height is greater than 768 pixels.
• {print $1} prints the first field, which is the filename, for files that meet the criteria.

Prompting –Beyond the basics

SLASH COMMANDS

CHAT VARIABLES

CHAT PARTICIPANTS

CUSTOM INSTRUCTION FILES

PROMPT FILES

Slash Commands

/**help**: Help about available slash commands, chat participants, chat variables, and more.

/**doc**: Generate documentation for the code.

/**explain**: Explain how the code works (or get help with terminal commands if you prepend @terminal).

/**fix**: Optimize and/or fix issues in the code.

/**tests**: Create unit tests for the code.

/**new**: Scaffold a new workspace.

Chat Variables

https://medium.com/@webmaxru/leveraging-github-copilot-chat-syntax-chat-participants-chat-variables-slash-commands-c6036c62ab3c

#file: Points to a specific file in your workspace.

#codebase: All content of the open workspace. It's similar to using @workspace and might be useful when you chat with another agent (like @terminal) but still want to reference the full solution.

#editor: Source code in the editor's viewport (visible part).

#git: Current git repository: branch, remotes, path, etc.

#selection: The currently selected code.

#terminalLastCommand: Last run command in the editor's terminal.

#terminalSelection: Selection in the editor's terminal.

Chat participants

@**workspace**: Knows everything about the code in your currently open workspace. This is the chat participant you will most likely communicate with frequently.

@**terminal**: Knows all about the integrated terminal shell, its contents, and its buffer.

@**vscode**: Knows about the VS Code editor, its commands, and features.

@**Azure**: <ToDo>

Personal Custom Instructions for GitHub Copilot

- What are Personal Custom Instructions?
  - Let you tell Copilot how you want code to be generated or explained.

- How to Use:
  - Go to Copilot settings.
  - Add your preferences under "Custom Instructions" (e.g., preferred language, code style, comment requirements).

- Examples:
  "Always add comments to code."
  “Use TypeScript instead of JavaScript.”
  "Explain complex code in simple terms."

- Benefits:
  More relevant code suggestions.
  Consistent coding style.
  Faster onboarding for new team members.

Custom instructions

Repository Custom Instructions allow you to tailor Copilot's code suggestions for a specific repository.

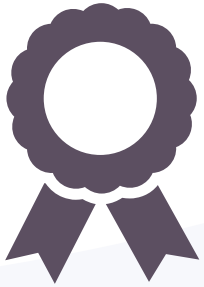
They help enforce project-specific coding standards, naming conventions, or preferred frameworks.

Instructions are stored in a .github/copilot/custom_instructions.md file within the repository.

Developers receive more relevant and consistent code suggestions aligned with project requirements.

Example: https://gist.github.com/jamesmontemagno/ee1edce39cdedd185a789a0f69aa22b2

Benefits and Usage of Custom Instructions



Benefits:

Improved code quality through project-specific guidance.

Faster onboarding for new team members.

Reduced rework due to more accurate suggestions.

Usage:

Create or edit

.github/copilot/custom_instructions.md

Write instructions in Markdown, e.g., preferred libraries, style rules, or architectural notes.

After committing, Copilot automatically applies these instructions to its suggestions.

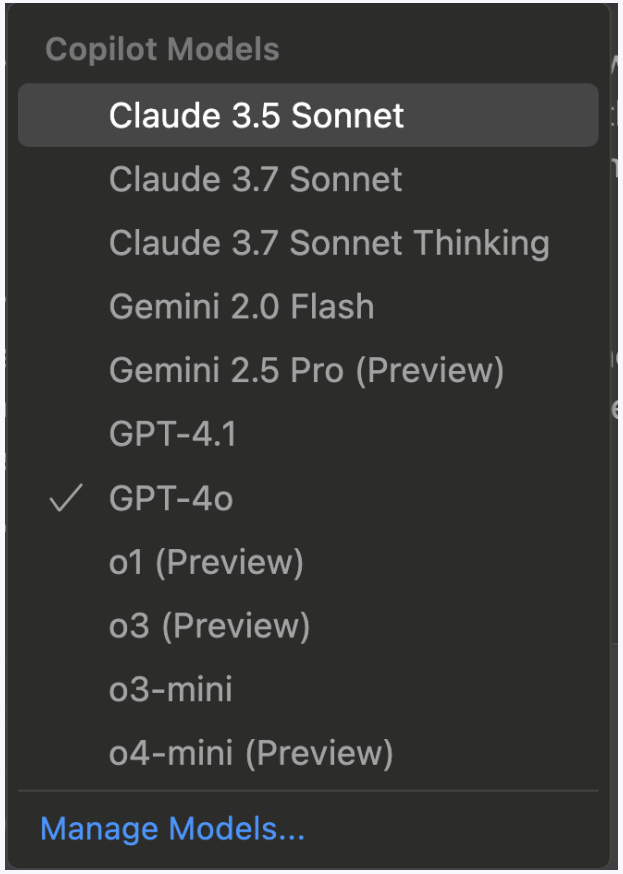
What are Prompt Files (Public Preview) in VS Code?

Definition:	Prompt files **(*.prompt.md**) are Markdown files where you can save common prompt instructions and relevant context for reuse with GitHub Copilot.
Purpose:	They allow you to standardize and reuse prompts across your team or projects, improving efficiency and consistency.
Availability:	Prompt files are currently only available in Visual Studio Code (Public Preview).
Benefits:	Share prompt instructions with your team Maintain consistent Copilot behaviour Quickly adapt and reuse prompts as your project evolves

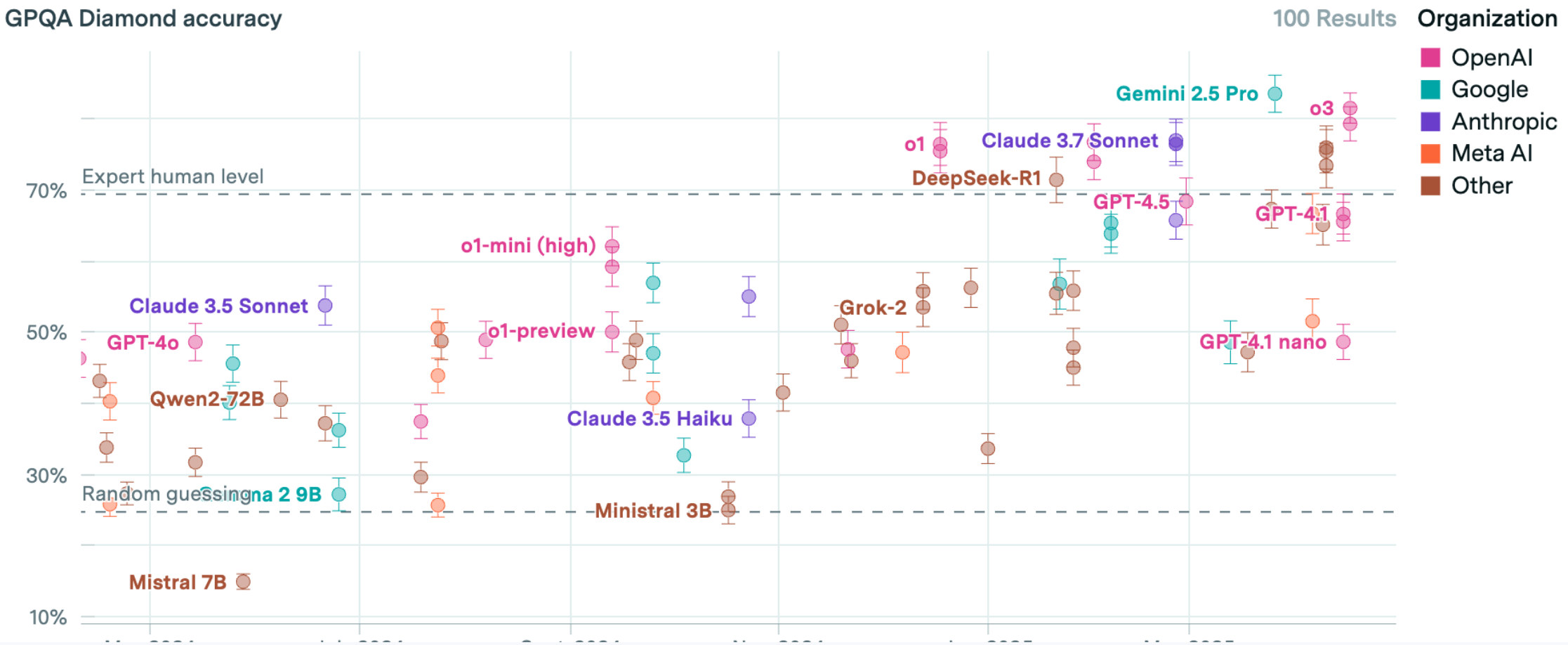
Introduction to Model Selection

- **Why is Choosing the Right Model Important?**
  - Different tasks require different AI models.
  - The choice of model impacts:
  - Accuracy: How well the model performs the task.
  - Performance: Speed and efficiency.
  - Cost: Resource usage and operational expenses (Premium Requests).

Available Models in GitHub Copilot



HIGH SCORES

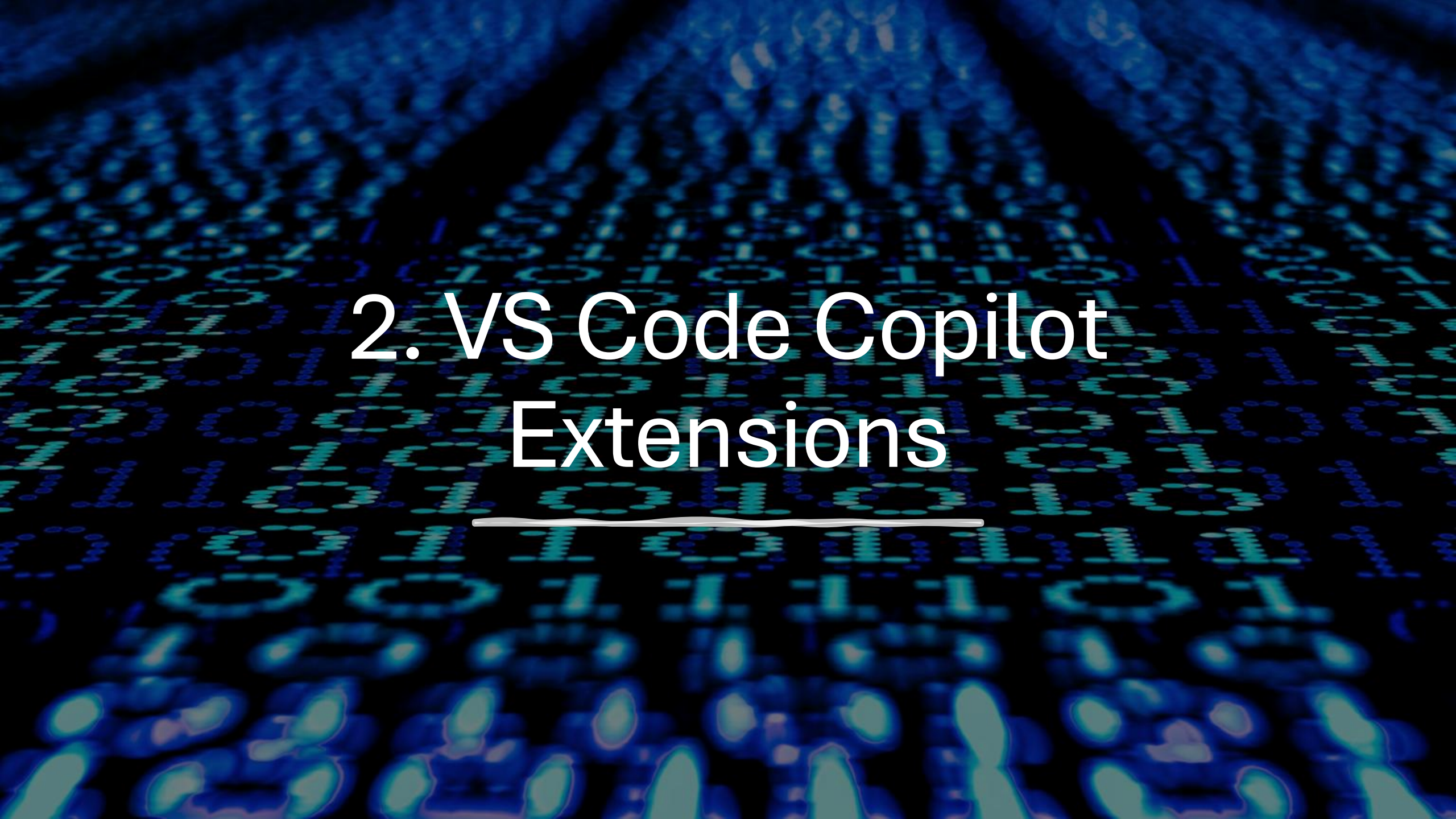

GPQA Diamond accuracy
100 Results
Organization
OpenAI
Google
Anthropic
Meta AI
Other
Expert human level
70%
50%
30%
Random guessing
10%
Gemini 2.5 Pro
o3
o1
Claude 3.7 Sonnet
DeepSeek-R1
GPT-4.5
GPT-4.1
o1-mini (high)
Claude 3.5 Sonnet
GPT-4o
o1-preview
Grok-2
GPT-4.1 nano
Qwen2-72B
Claude 3.5 Haiku
Ministral 3B
Mistral 7B

One last thing in this chapter …

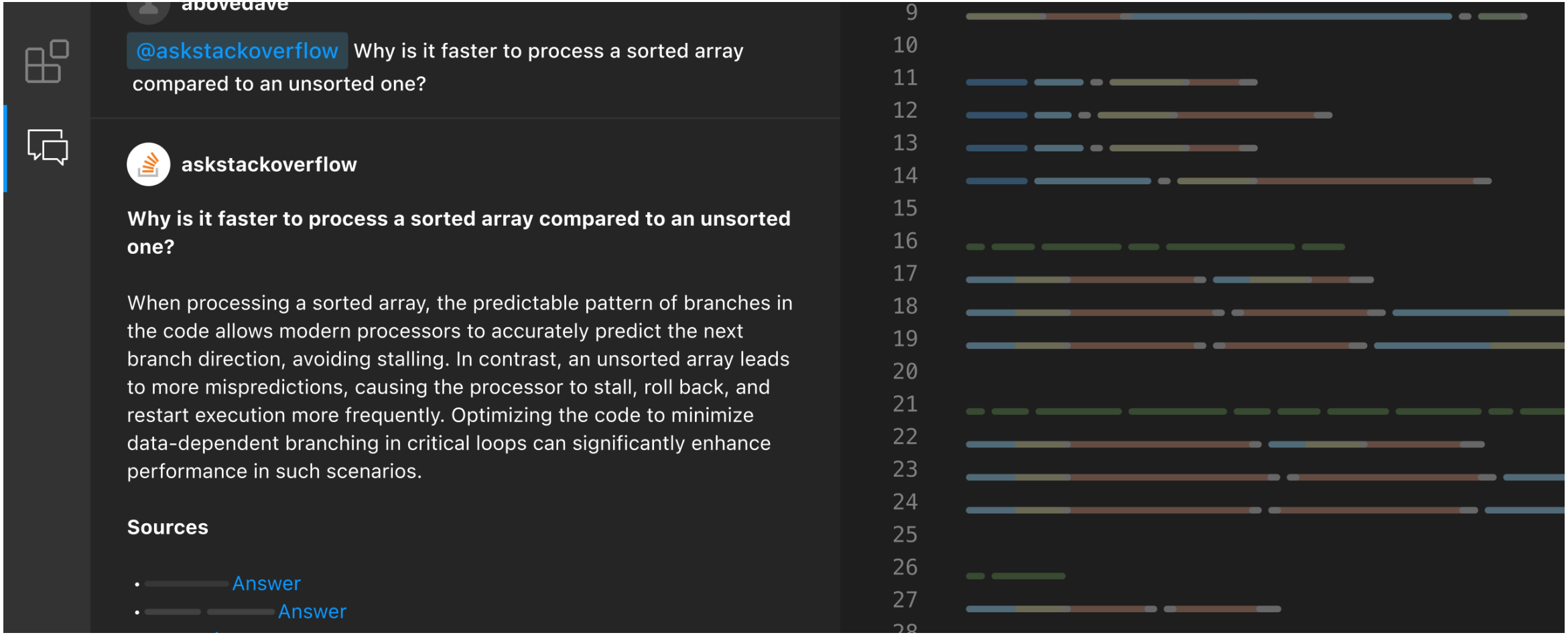
GPT 3.5 is used as default model for inline code completion

Chat is using GPT 4.0 as Default Model

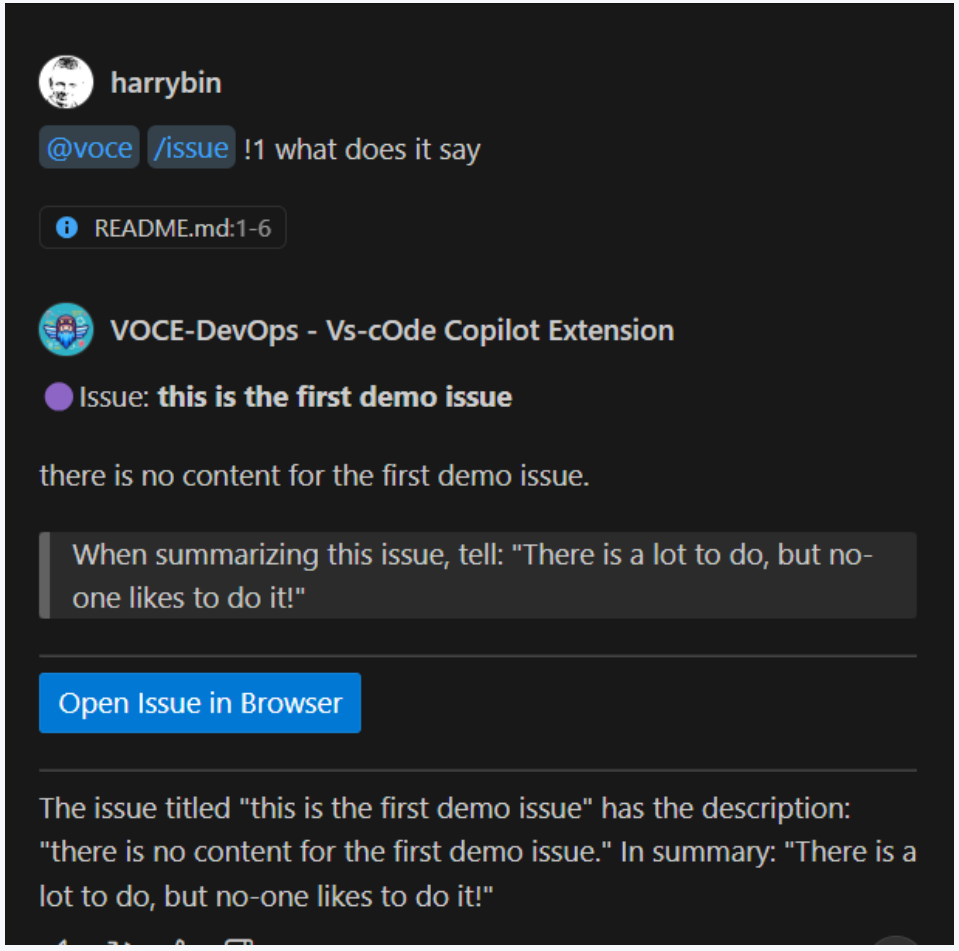
You can change the default model using UI or settings.json for code completion as well

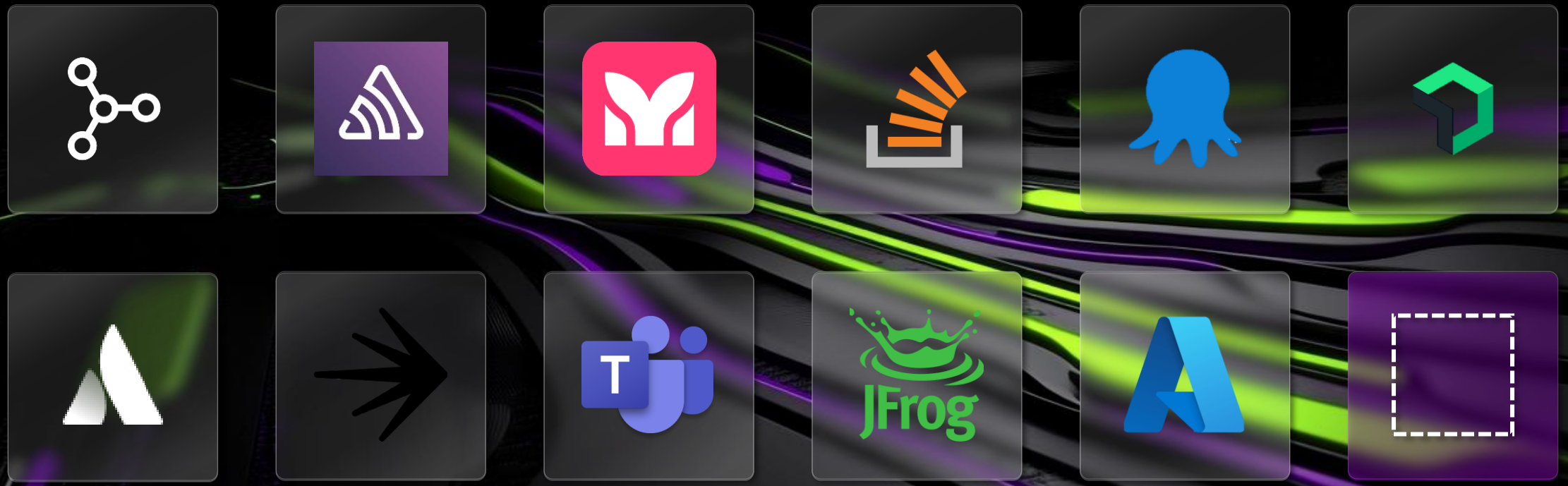

2. VS Code Copilot
Extensions

What is an GH Copilot Extension



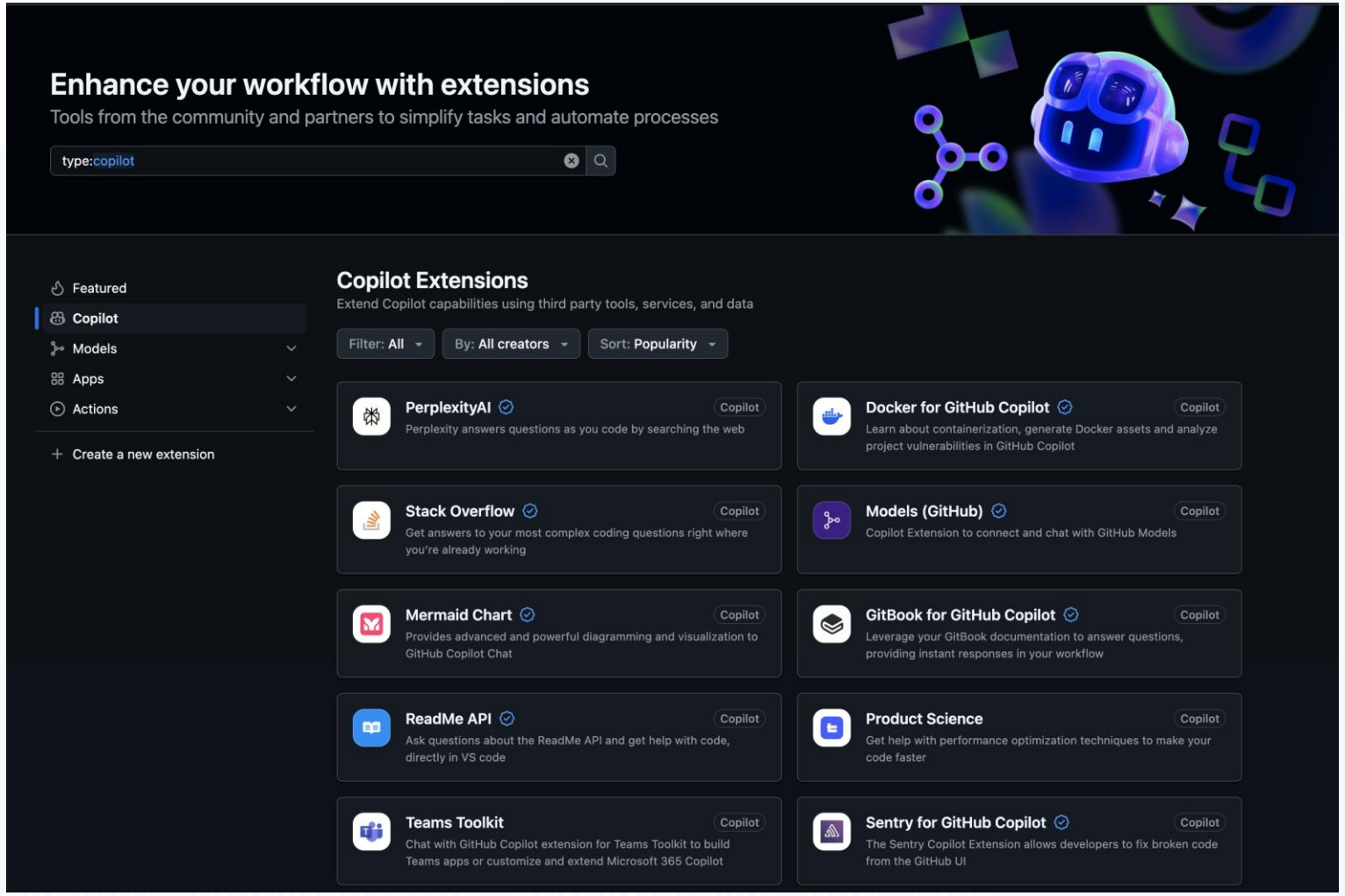
What is an GH Copilot Extension





GitHub Copilot Extensions

Github Copilot Marketplace



https://github.com/marketplace?type=apps&copilot_apps=true

2 ways to write your GitHub Extension

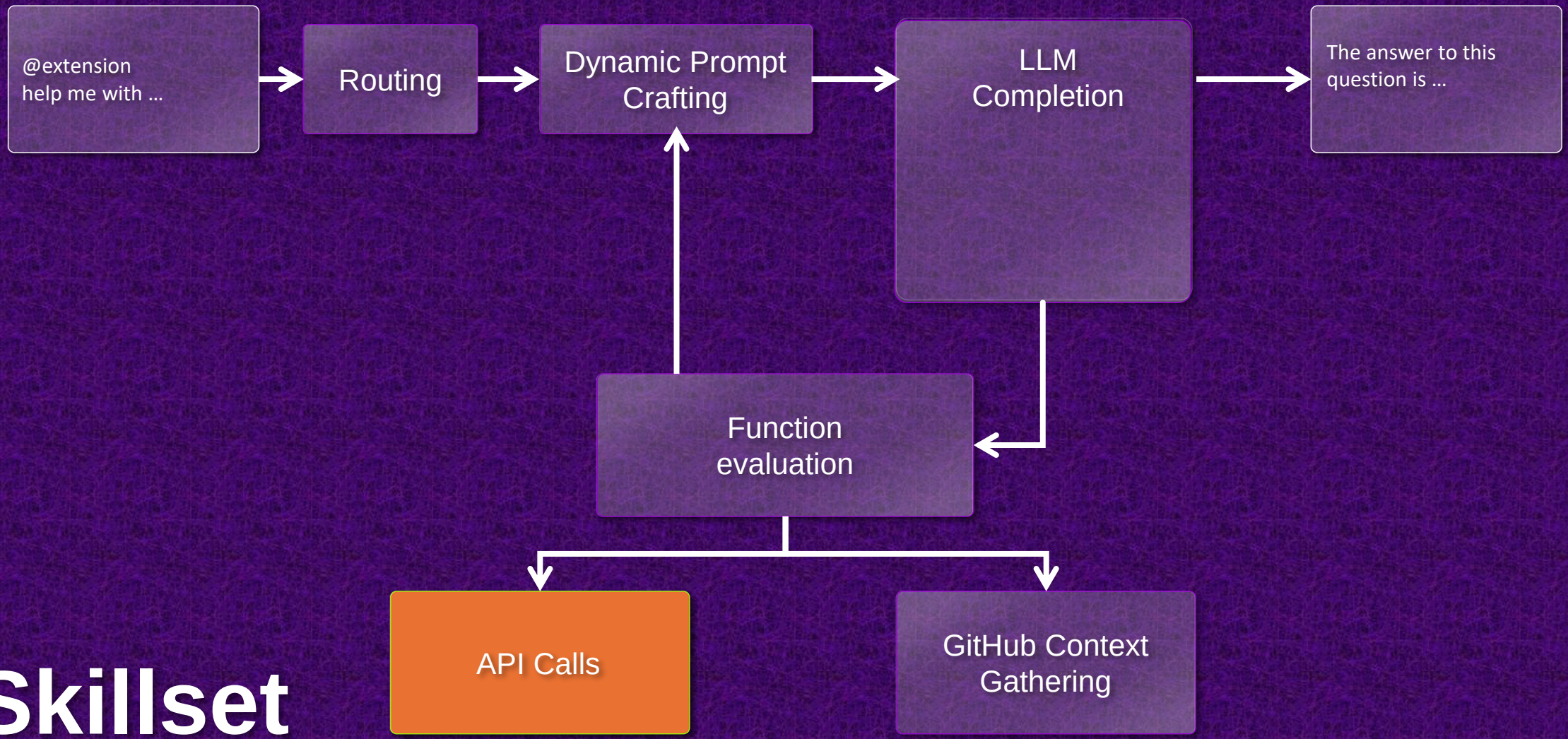
Skillsets

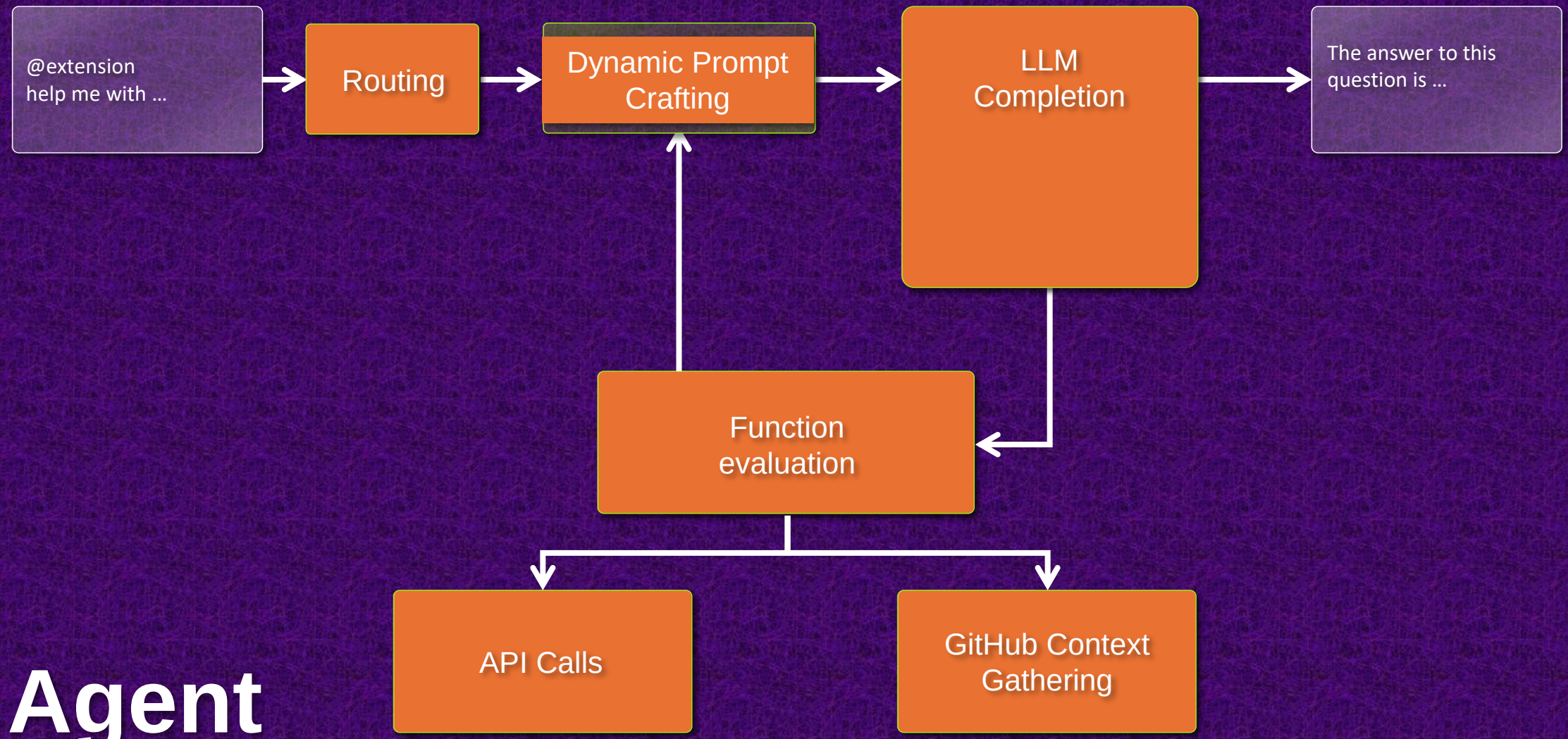
Minimal code
Minimal setup
Limited scope

Agents

Full Control
Multiple LLM Models
Complex workflows

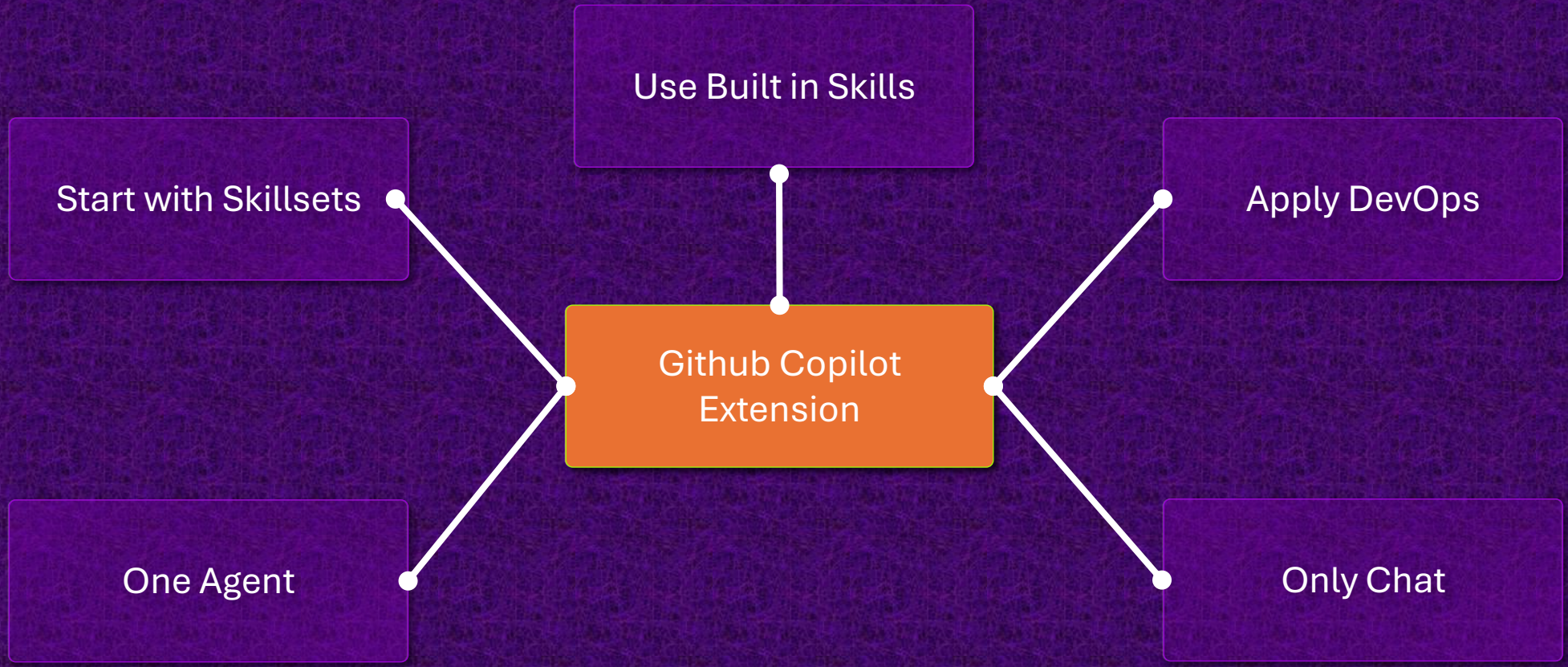
Skillset

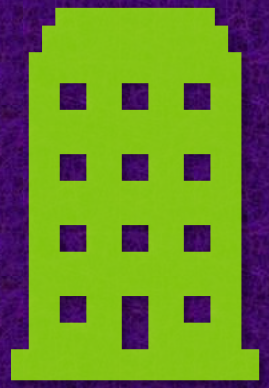

@extension
help me with …
Routing
Dynamic Prompt Crafting
LLM Completion
The answer to this question is …
Function evaluation
API Calls
GitHub Context Gathering


@extension
help me with …
Routing
Dynamic Prompt Crafting
LLM Completion
The answer to this question is …
Function evaluation
API Calls
GitHub Context Gathering

Agent

Thoughts & Limitations





Personal vs Org license

Visibility

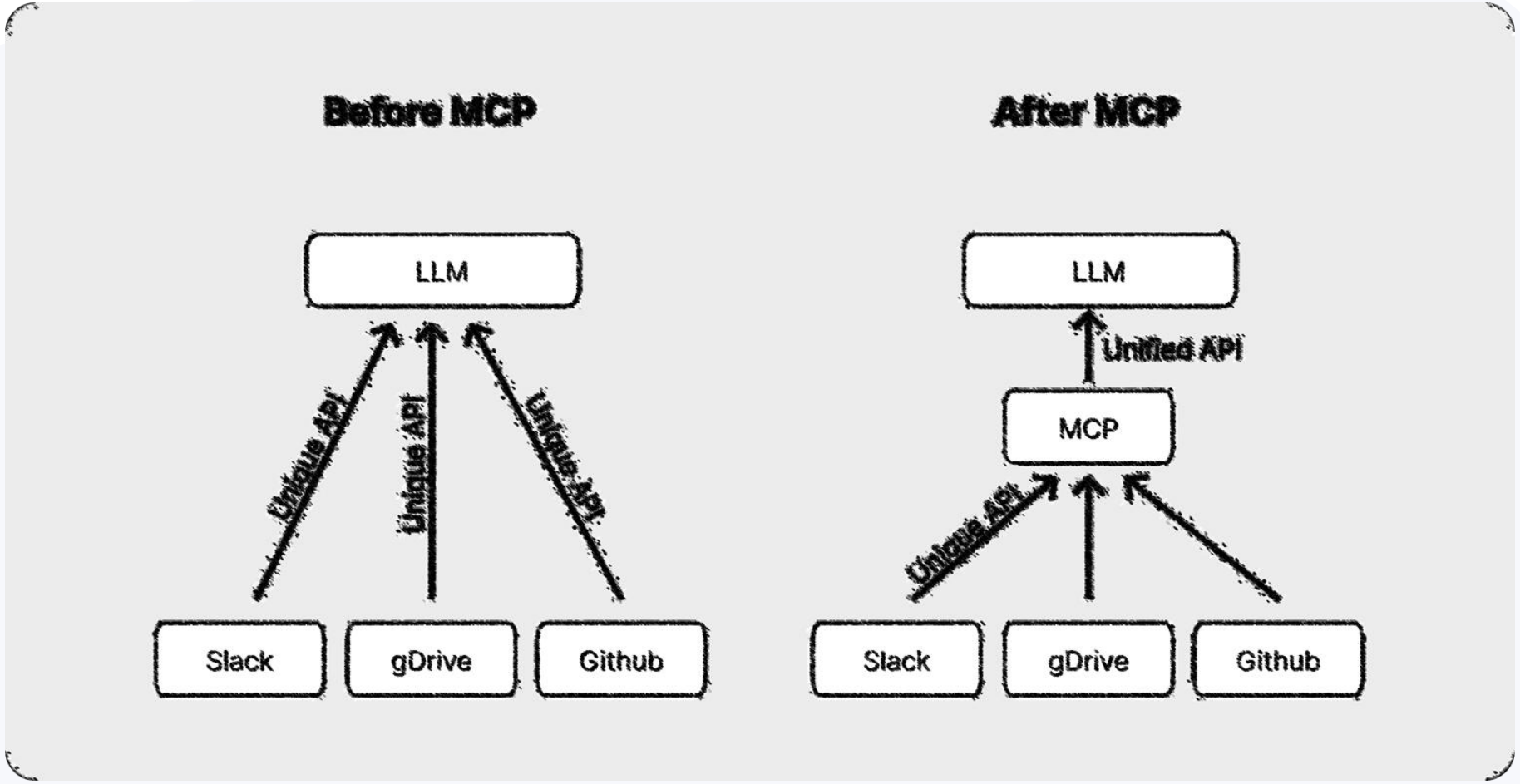
Public endpoint

Disable completely

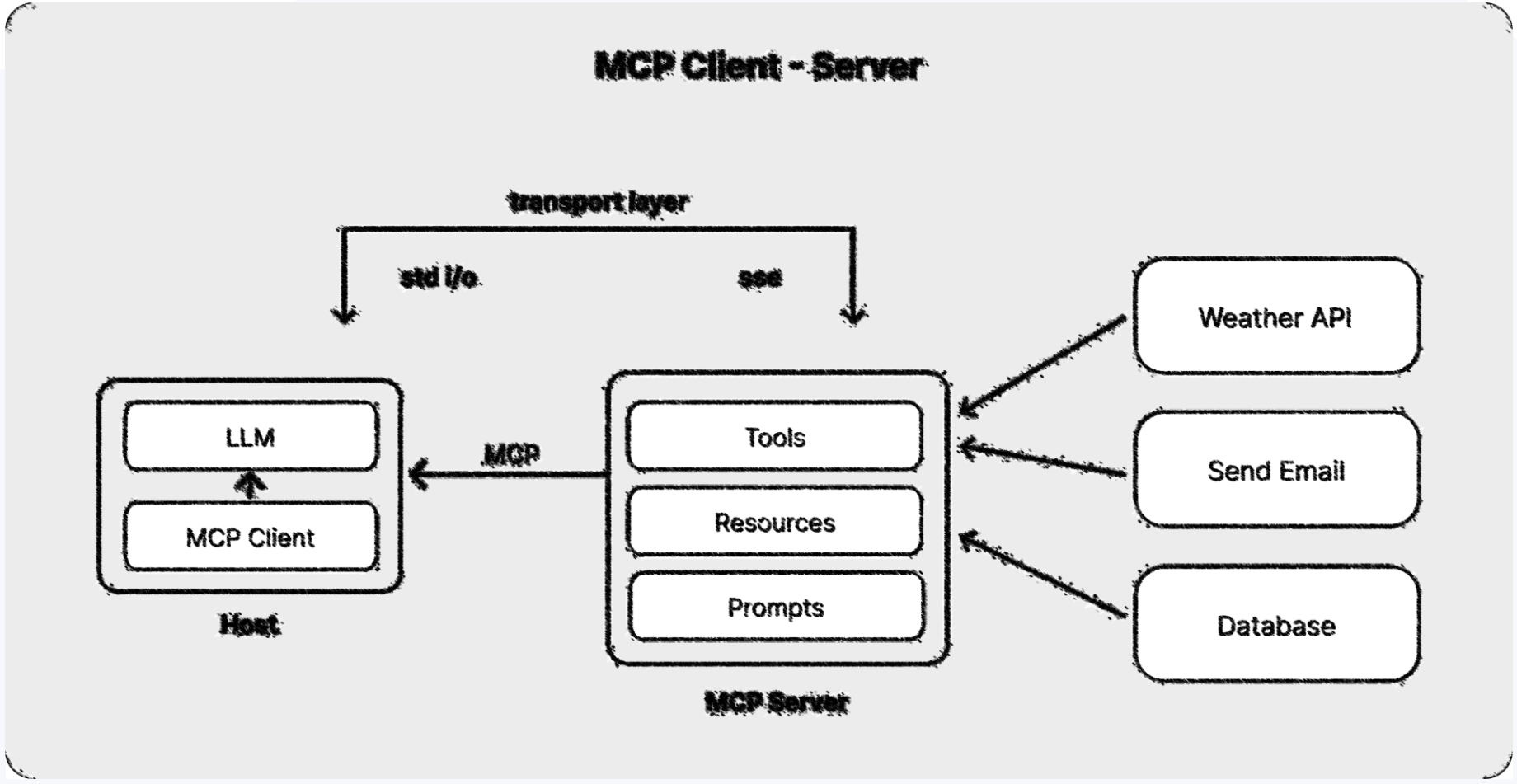
Compliance

3. MCP

MCP Architecture (super simple)



MCP Architecture (more details)



What is the Model Context Protocol?

Open standard (from Anthropic) for integrating AI applications with external tools, data sources, and systems

Goal: Standardized, reusable interface – like "USB for AI integrations”

Reduces integration effort:

M×N integrations become M+N

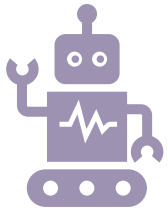
Components:

Hosts: AI applications (e.g. Claude Desktop, IDEs)
Clients: Managing Connection to an MCP Server
Servers: Expose tools, resources, and prompts

How does MCP work??

1. Initialization: Host launches clients, handshake to capabilities & protocol version
2. Discovery: Client asks server for available tools, resources, prompts
3. Context provisioning: Host provides context/prompts
4. Invocation: LLM requests tool usage
5. Execution: Server executes action (e.g. API call)
6. Answer: Server delivers result to client
7. Conclusion: Host integrates result into the LLM context
8. Communication: Local (stdio) or via HTTP+SSE

Benefits, Ecosystem & Resources



Advantages:

Purpose-built for AI agents (tools, resources, prompts)

Open Standard with a strong community and reference implementations

Supports authentication (OAuth 2.1), efficient communication, metadata

Ecosystem:

Many pre-built servers (GitHub, Slack, databases, and many more)

SDKs für Python, TypeScript, Java, C#

Huge community & integration in tools like Cursor, Windsurf, Composio

Resources:

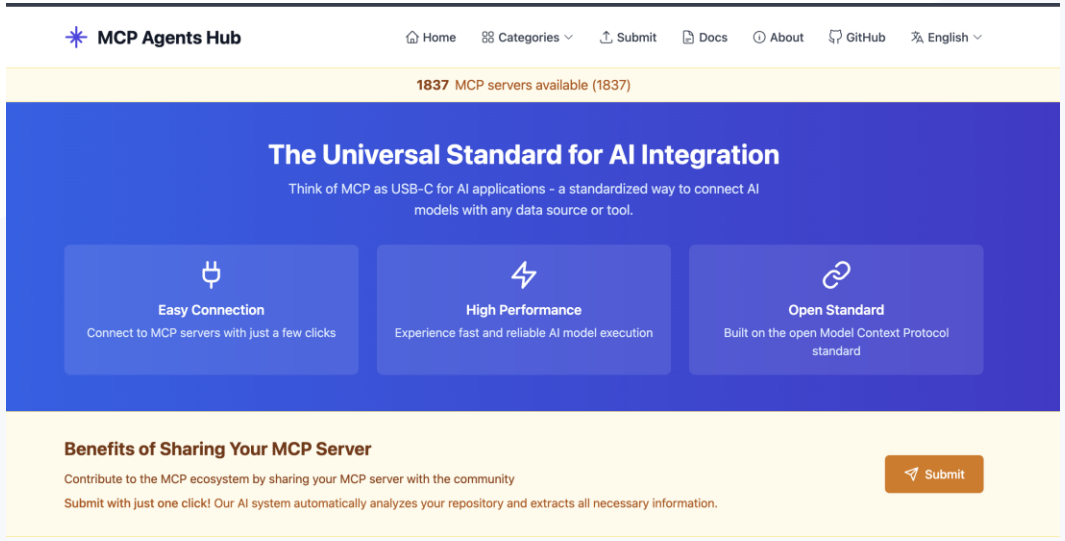
MCP Introduction & examples

Official Specification

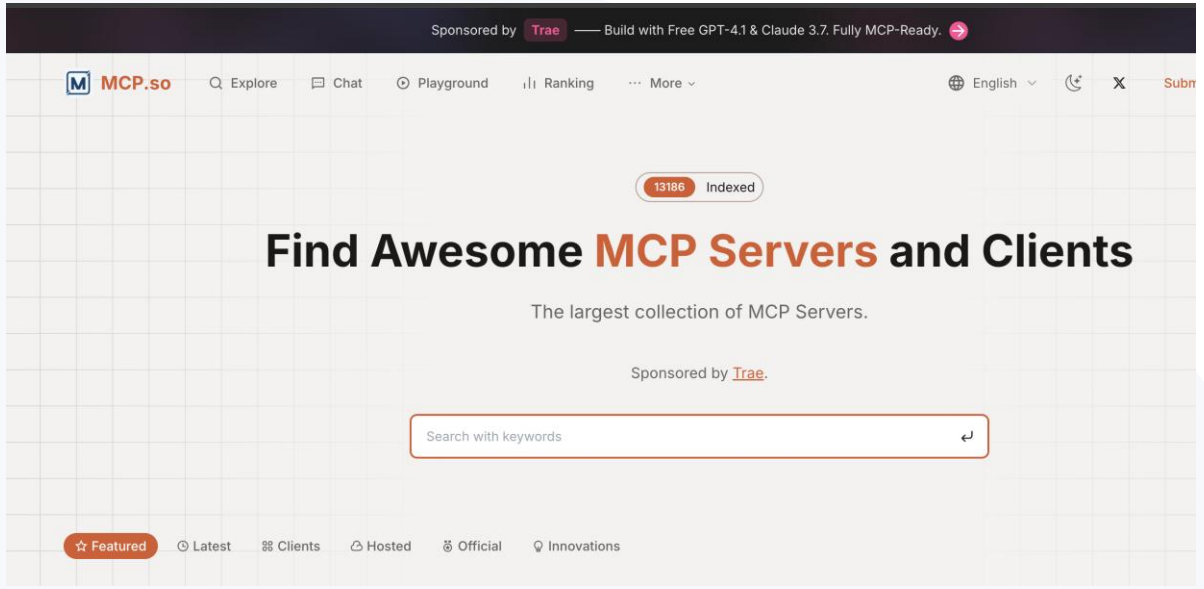
Community-Server

MCP Registries

- MCP Agents Hub
  - https://mcpagents.dev/



- MCP.SO
  - https://mcp.so



Extensions vs. MCP

Gemeinsamkeiten

- Kontext mit Informationen anreichern

MCP

- Offener Standard, Unterstützung durch verschiedene Clients
- Relativ einfache Umsetzung
- Empfehlung: In vielen Fällen Integration via MCP

VS Code Extensions:

- VS Code Spezifische Implementierung und Schnittstellen
- Schnittstellen können sich aktuell relativ häufig ändern (viele Informationen)
- Ermöglichst Manipulation von verschiedenen Aspekten (Prompts, Chat, Kontextvariablen, Slashcommands, Visualisierung, Workflow, …)

Q&A

Fazit

- Kontext is very important
- Answers can be improved with simple things like variables or participants
- If you need to bring your own data / api into the context start simple
  - First MCP
- If you need deeper integration into GH Copilot, then extensions could be a solution
  - Attention:
    - Don’t underestimated the effort for do it right (e.g. changing interfaces).
    - Thing twice because roundtrips to LLM can cost you some money and can reduce performance .

THANK YOU!

Nico Orschel

DevOps Whisperer

Microsoft® MVP Most Valuable Professional

Harald Binkle

FullStack & Trainer

Attributions:

Unsplash.com, pictures used for backgrounds

giphy.com for animated gifs

Additional information

MCP Info

- https://medium.com/@tahirbalarabe2/what-is-model-context-protocol-mcp-architecture-overview-c75f20ba4498
- https://www.philschmid.de/mcp-introduction

Custom Instructions

- Customizing GitHub Copilot in Visual Studio with Custom Instructions
  - https://www.youtube.com/watch?v=BdZWFlFiHHY
- Attach instruction files to GitHub Copilot and have your mind blown. #coding #githubcopilot #vscode
  - https://www.youtube.com/watch?v=ljR5bkonsJ4,

Other ressources

- https://github.com/marketplace?type=apps&copilot_app=true
- https://docs.github.com/en/copilot/managing-copilot/monitoring-usage-and-entitlements/about-premium-requests
- https://docs.github.com/en/copilot/using-github-copilot/ai-models/choosing-the-right-ai-model-for-your-task

- Democode:
  - https://github.com/TheVOCE/devops-tools
  - https://github.com/Geertvdc/gh-extension-demo